토스페이먼츠 결제 연동 API 명세 (프론트엔드)

2026년 10월 19일 · @Someone

변경 요약

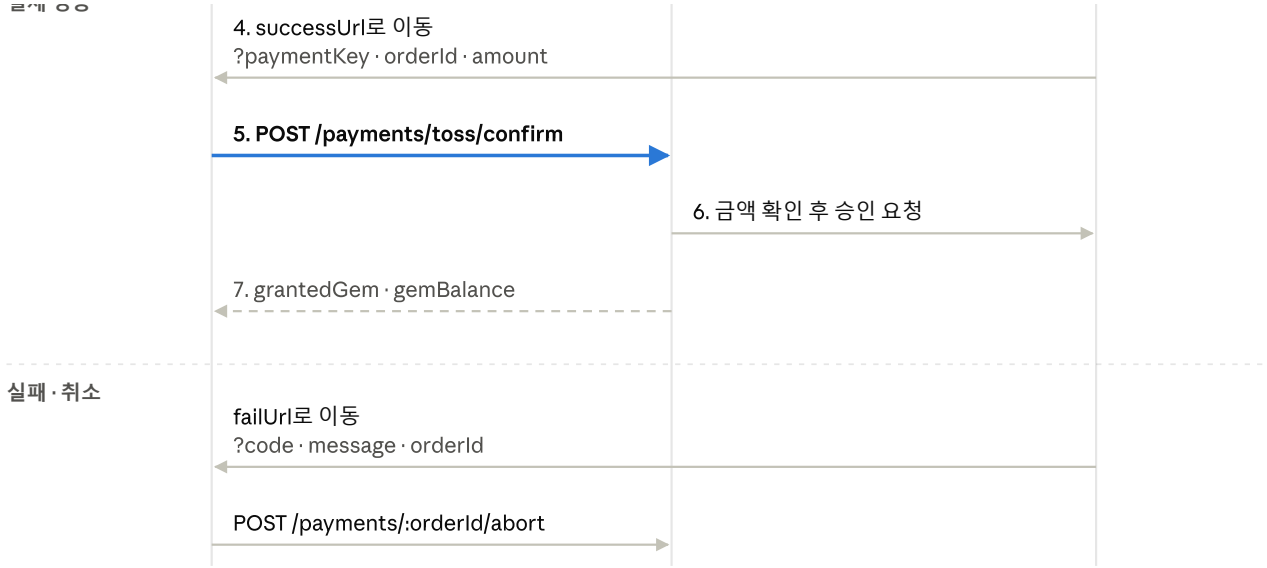
보석 결제가 카카오페이에서 토스페이먼츠로 바뀌어, 프론트는 결제 API 2개의 경로와 형식, 그리고 결제 관련 화면 4개를 고쳐야 합니다. 가장 큰 차이는 결제창을 서버가 아니라 **프론트가 토스 SDK로 직접 연다**는 점입니다.

구분	대상	할 일
신규	`@tosspayments/tosspayments-sdk`	패키지 설치. 클라이언트 키는 서버 응답에 들어 있어 환경변수는 필요 없음
수정	`POST /payments/kakaopay/ready`	`POST /payments/toss/ready`로 경로 변경, 응답 필드 변경
수정	`POST /payments/kakaopay/approve`	`POST /payments/toss/confirm`으로 경로 변경, 요청 필드 변경 (`pgToken` → `paymentKey`, `amount`)
수정	상점 페이지 구매 버튼	리다이렉트 URL로 이동하던 코드 → SDK `requestPayment()` 호출
수정	`/shop/payment/[orderId]/success`	쿼리 `pg_token` 대신 `paymentKey`, `orderId`, `amount`를 읽어 confirm 호출
수정	`/shop/payment/[orderId]/fail`	쿼리 `code`, `message`를 읽어 abort 호출 + 메시지 표시
삭제	`/shop/payment/[orderId]/cancel`	토스는 취소도 fail 페이지로 보내므로 더 이상 쓰지 않음
그대로	상품 목록, 환불, 결제 내역 API	변경 없음

결제 흐름

confirm을 불러야 결제가 완료되고 보석이 지급된다





프론트가 직접 부르는 API는 1번(ready), 5번(confirm), 실패 시 abort 세 개입니다. 6번 토스 승인은 백엔드가 처리합니다.

공통 규칙

결제 API는 모두 로그인이 필요합니다. `Authorization: Bearer <accessToken>` 헤더를 붙이고, 바디는 JSON(`Content-Type: application/json`)으로 보냅니다.

모든 응답은 아래 공통 형식입니다. 성공이면 `data`에 결과가 들어 있고, 실패면 `error.message`를 그대로 사용자에게 보여줘도 됩니다.

```
{ "success": true,  "data": { ... }, "error": null }
{ "success": false, "data": null,    "error": { "code": "INVALID_STATE",
"message": "결제 금액이 일치하지 않습니다" } }
```

HTTP	error.code	의미
400	`INVALID_INPUT`	필수 값 누락, 형식 오류 (예: `amount`를 문자열로 보냄)
401	—	토큰 없음 또는 만료
404	`NOT_FOUND`	상품·주문이 없거나 내 주문이 아님
409	`INVALID_STATE`	지금 상태에서 할 수 없는 요청 (금액 불일치, 이미 실패한 주문 등)
502	`PAYMENT_FAILED`	토스 승인·취소 실패 또는 토스 연결 실패

API 명세

1. 결제 준비 — `POST /payments/toss/ready` (수정)

주문을 만들고, 토스 결제창을 여는 데 필요한 값을 돌려줍니다. 이 단계에서는 돈이 빠지지 않습

니다.

요청

```
{ "productId": 3 }
```

필드	타입	필수	설명
`productId`	number	O	`GET /shop/products`의 `productId`

응답 `data`

```
{
  "orderId": "9b1d4f2e-3c5a-4e8b-a7d1-2f6c8e0b4a91",
  "orderName": "보석 30개 구매",
  "amount": 3000,
  "gemAmount": 30,
  "clientKey": "test_ck_...",
  "successUrl": "https://kiwi.site/shop/payment/9b1d4f2e-.../success",
  "failUrl": "https://kiwi.site/shop/payment/9b1d4f2e-.../fail"
}
```

필드	타입	설명
`orderId`	string	주문 번호 (UUID). SDK `orderId`에 그대로 넣음
`orderName`	string	상품명. SDK `orderName`에 그대로 넣음
`amount`	number	결제 금액(원). SDK `amount.value`에 넣음. 상점 화면에는 보여주지 않음
`gemAmount`	number	지급될 보석 개수 (기본 + 보너스)
`clientKey`	string	토스 클라이언트 키. `loadTossPayments()`에 넣음
`successUrl`	string	결제 성공 시 돌아올 주소. SDK에 그대로 넣음
`failUrl`	string	결제 실패·취소 시 돌아올 주소. SDK에 그대로 넣음

기존 응답에 있던 `pcRedirectUrl`, `mobileRedirectUrl`, `appRedirectUrl`은 없어졌습니다.

HTTP	메시지	상황
400	`productId: ...`	`productId` 누락
404	판매 중인 상품이 아닙니다	없거나 판매 종료된 상품
409	이용할 수 없는 계정입니다	비활성 회원

2. 결제 승인 `POST /...` (승격)

2. 결제 승인 — `POST /payments/toss/confirm` (수정)

success 페이지에서 부릅니다. 서버가 금액을 확인하고 토스에 승인을 요청한 뒤 보석을 지급합니다. **이 API를 부르지 않으면 결제가 완료되지 않습니다** (토스는 인증 후 10분 안에 승인해야 함).

요청

```
{
  "orderId": "9b1d4f2e-3c5a-4e8b-a7d1-2f6c8e0b4a91",
  "paymentKey": "tgen_20261019...",
  "amount": 3000
}
```

필드	타입	필수	설명
`orderId`	string	O	successUrl 쿼리의 `orderId`
`paymentKey`	string	O	successUrl 쿼리의 `paymentKey`
`amount`	**number**	O	successUrl 쿼리의 `amount`. 쿼리는 문자열이므로 `Number()`로 바꿔서 보냄

기존 요청의 `pgToken`은 없어졌습니다.

응답 `data` (형식 기존과 동일)

```
{ "orderId": "9b1d4f2e-...", "grantedGem": 30, "gemBalance": 130 }
```

필드	타입	설명
`orderId`	string	주문 번호
`grantedGem`	number	이번에 지급된 보석
`gemBalance`	number	지급 후 보석 잔액

이미 승인된 주문을 다시 보내면(새로고침, 중복 호출) 보석을 다시 주지 않고 같은 결과를 200으로 돌려줍니다.

HTTP	메시지	상황	프론트 처리
400	`amount: ...` 등	필수 값 누락, `amount`가 숫자가 아님	요청 코드 수정
404	결제를 찾을 수 없습니다	없는 주문 또는 다른 회원의 주문	상점으로 돌려보냄
409	결제 금액이 일치하지 않습니다	쿼리 `amount`가 주문 금액과 다름. 돈은 빠지지 않음	오류 표시 후 상점으로
409	승인할 수 없는 결제입니다	이미 실패·환불된 주문	처음부터 다시 구매

502	토스페이먼츠 결제 처리에 실패했습니다	토스가 승인을 거절 (한도 초과, 10분 초과 등). 주문은 실패 처리됨	처음부터 다시 구매
502	토스페이먼츠에 연결할 수 없습니다	토스 연결 실패 또는 응답 지연. 서버가 토스에 결제를 조회해 승인됐으면 자동 취소하므로 보석 없이 돈만 빠지지 않음	처음부터 다시 구매

3. 결제 중단 — `POST /payments/{orderId}/abort` (기존 그대로)

fail 페이지에 도착했거나 SDK가 오류를 던졌을 때 부릅니다. 바디는 없고, 성공하면 `data`가 `null`입니다.

- 대기 중인 주문만 실패로 바꿉니다. 이미 결제된 주문은 바꾸지 않고 200을 줍니다. 그래서 여러 번 불러도 안전합니다.
- 404: 없는 주문이거나 다른 회원의 주문

화면별 구현 가이드

아래 코드는 Next.js(App Router) 기준 예시입니다. `api`는 인증 헤더를 붙이는 기존 HTTP 클라이언트로 바꿔 쓰면 됩니다.

```
npm install @tosspayments/tosspayments-sdk
```

상점 페이지 (구매 버튼)

기존의 PC·모바일 리다이렉트 분기는 지웁니다. SDK가 기기에 맞는 결제창을 알아서 엽니다.

```
import { loadTossPayments, ANONYMOUS } from "@tosspayments/tosspayments-sdk";

async function buy(productId: number) {
  const { data } = await api.post("/payments/toss/ready", { productId });

  const toss = await loadTossPayments(data.clientKey);
  const payment = toss.payment({ customerKey: ANONYMOUS });

  try {
    await payment.requestPayment({
      method: "CARD",
      amount: { currency: "KRW", value: data.amount },
      orderId: data.orderId,
      orderName: data.orderName,
      successUrl: data.successUrl,
      failUrl: data.failUrl,
    });
  } catch (e) {
    // PC에서 결제창을 닫으면 페이지 이동 없이 여기로 온다
    await api.post(`/payments/${data.orderId}/abort`);
```

```
  }
}
```

- 구매 버튼은 요청 중 비활성화해 중복 주문을 막습니다.
- `amount`는 SDK에만 넘기고, 화면에는 지금처럼 보석 개수만 보여줍니다.

success 페이지 `/shop/payment/[orderId]/success`

토스가 `?paymentKey=...&orderId=...&amount=...`를 붙여 보내줍니다. 이 값을 그대로 confirm에 넘깁니다.

```
"use client";
import { useEffect, useRef, useState } from "react";
import { useSearchParams } from "next/navigation";

export default function PaymentSuccessPage() {
  const params = useSearchParams();
  const sent = useRef(false);
  const [result, setResult] = useState(null);
  const [error, setError] = useState(null);

  useEffect(() => {
    if (sent.current) return;   // Strict Mode 중복 호출 방지
    sent.current = true;

    api.post("/payments/toss/confirm", {
      orderId: params.get("orderId"),
      paymentKey: params.get("paymentKey"),
      amount: Number(params.get("amount")),
    })
      .then((res) => setResult(res.data))       // grantedGem, gemBalance
      .catch((e) => setError(e.response?.data?.error?.message ?? "결제 승인에 실패
했습니다"));
  }, []);

  // 로딩 → 성공(지급 보석, 잔액) 또는 실패(error) 화면
}
```

- 승인이 끝날 때까지 로딩 화면을 보여주고, 뒤로 가기를 막거나 안내합니다.
- 성공하면 전역 보석 잔액 상태를 `gemBalance`로 갱신합니다.
- 새로고침해서 다시 불려도 보석은 한 번만 지급됩니다.

fail 페이지 `/shop/payment/[orderId]/fail`

토스가 `?code=...&message=...&orderId=...`를 붙여 보내줍니다. 사용자가 취소한 경우도 여기로 옵니다.

```
useEffect(() => {
  api.post(`/payments/${orderId}/abort`);   // orderId는 경로 파라미터
```

```
}, []);

const code = params.get("code");
const message = code === "PAY_PROCESS_CANCELED"
  ? "결제를 취소했습니다"
  : params.get("message");
```

- 메시지와 함께 "상점으로 돌아가기" 버튼을 둡니다.

cancel 페이지 `/shop/payment/[orderId]/cancel` (삭제)

토스에는 따로 취소 주소가 없어 더 이상 호출되지 않습니다. 남겨두고 싶다면 fail 페이지로 리다이렉트해도 됩니다.

변경 없는 API

아래 API는 경로와 형식이 그대로입니다. 환불은 서버 내부에서만 토스 취소로 바뀌었습니다.

API	용도	응답 `data`
`GET /shop/products`	상품 목록	`[{ productId, name, gemAmount, price, salePrice }]`
`POST /payments/{orderId}/refund`	환불 (결제 후 7일 이내, 보석을 쓰지 않았을 때만)	`{ orderId, revokedGem, gemBalance }`
`GET /payments/my?page=0&size=20`	내 결제 내역 (결제 완료·환불 건만)	`[{ orderId, amount, productName, grantedGem, status, refundable, paidAt, canceledAt, createdAt }]`

체크리스트와 테스트

백엔드에 토스 **테스트 키**(`test_ck_` / `test_sk_`)가 설정돼 있으면 실제 승인은 되지만 돈은 빠지지 않습니다. 결제창에서 아무 카드나 골라 진행하면 됩니다.

- [ ] SDK 설치
- [ ] 구매 버튼: `/payments/toss/ready` 호출 후 `requestPayment()`로 결제창 열기
- [ ] 기존 리다이렉트 URL(`pcRedirectUrl` 등) 처리 코드 삭제
- [ ] success 페이지: `paymentKey`, `orderId`, `amount`(숫자)로 `/payments/toss/confirm` 호출
- [ ] success 페이지: 성공 시 보석 잔액 갱신, 실패 시 `error.message` 표시
- [ ] fail 페이지: abort 호출 + 취소/실패 메시지 표시
- [ ] cancel 페이지 삭제
- [ ] 테스트 PG에서 결제 성공 → 보석 지급 확인

- [ ] 테스트: PC에서 결제 성공 → 보석 지급 확인
- [ ] 테스트: 모바일 브라우저에서 결제 성공
- [ ] 테스트: 결제창을 닫거나 취소 → fail 페이지 또는 catch에서 abort 호출
- [ ] 테스트: success 페이지 새로고침 → 보석이 두 번 지급되지 않음
- [ ] 테스트: 결제 내역에서 환불 → 보석 회수 확인

참고: 토스페이먼츠 결제창 SDK v2 가이드